# Mathematical Language of Deep Learning | 1

**Chapter Abstract.** Deep learning is often introduced through software: load data, build a model, train it, and report accuracy. This chapter slows down that workflow and names the mathematical objects underneath it for readers who have not yet studied machine learning or programming. A small handwritten-digit example is used throughout to define inputs, labels, linear-algebra shape checks, parameterized functions, predictions, losses, empirical risk, layers, gradients, and generalization. Together, these objects give the precise vocabulary for the applied chapters that follow.

## 1.1 Running Example and Goals

An applied deep-learning project begins by choosing what a system should predict. For the handwritten-digit example in this chapter, the system receives an image and returns a label. The same mathematical question appears in larger settings: how can examples be used to choose a function that makes useful predictions on future cases? Deep learning answers this question with large parameterized functions trained from data.

The running example is a compact handwritten-digit classifier: a grayscale image must be labeled 0 or 1. Even this reduced problem contains the core objects. An input is represented by numbers, a model turns those numbers into a prediction, a loss measures the error, and training changes the parameters to reduce average loss. The same chain of ideas appears in larger classifiers, language models, retrieval systems, and generative models [67, 124].

To keep the notation concrete, we will often use a two-feature teaching example. Let

$$x = (x_1, x_2) = (0.20, 0.80).$$

The two numbers form a compact numerical stand-in for the idea that an image becomes an array of numbers. For a raw image, the feature list may contain one coordinate per pixel; here $x_1$ and $x_2$ should be read as summary features computed from many pixels, chosen only to keep the arithmetic visible. A simple score for this input might be

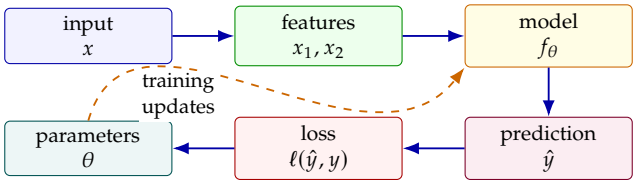


**Figure 1.1:** The running workflow for the chapter. Data provide inputs and labels, the model produces predictions, the loss scores those predictions, and training changes the parameters.

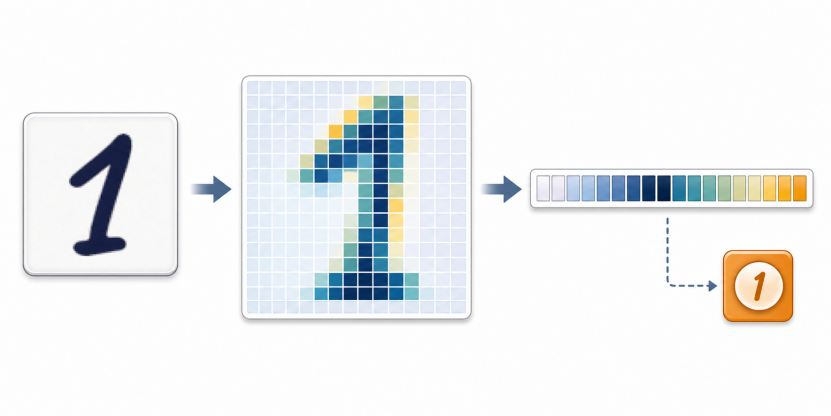

**Figure 1.2:** A handwritten digit can be represented as pixel values, flattened into an ordered feature list $x$, and paired with a label $y$. The chapter's two-feature example keeps this same data-to-numbers idea while making the arithmetic small enough to inspect by hand.

$$z = w_1 x_1 + w_2 x_2 + b,$$

where $w_1$, $w_2$, and $b$ are adjustable quantities. If $w_1 = 1.50$, $w_2 = -0.50$, and $b = 0.10$, then

**Pause and predict**

Before reading the arithmetic, identify which term pushes the score upward and which term pushes it downward. Which contribution has larger magnitude? If this example needed a larger score, would increasing $w_1$, increasing $w_2$, or increasing $b$ move the score in the right direction?

$$z = 1.50(0.20) - 0.50(0.80) + 0.10 = 0.00.$$

Here $x_1$ and $x_2$ are observed from the example. The quantities $w_1$, $w_2$, and $b$ are the ones training may change, and $z$ is the score produced by combining the two.

Inputs are observed. Parameters are chosen by training. The model output depends on both.

The definitions below stay tied to problems that show up in practice. In the digit example, we need a function to say what the model outputs, shapes to catch data-handling mistakes, a loss to say what training minimizes, gradients to explain how updates move, and generalization to explain why training accuracy alone can mislead. The formal language gives names to the objects that applied deep-learning work already uses.

Later chapters add tensors, convolution, backpropagation, transformers, retrieval, generative models, and reinforcement learning, but the basic objects remain the same: examples, labels, functions, parameters, losses, optimization procedures, and evaluation data. For a complementary introductory treatment of neural networks, see Nielsen [156]; for a broader reference, see Goodfellow, Bengio, and Courville [67].

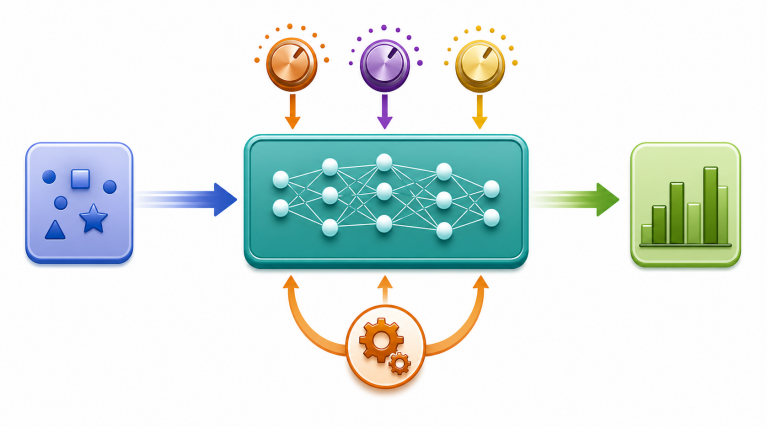

**Figure 1.3:** A parameterized model sends an input $x$ through a function $f_\theta$ to produce a prediction $\hat{y}$. Training changes the parameter collection $\theta$, so it changes which function in the family is used.

## 1.2 Deep Learning as Parameterized Functions

The central mathematical object in deep learning is a function. In ordinary language, a function is a rule that takes an input and produces an output. In deep learning, the input might be an image, a sound waveform, a sentence, a query, or the current state of a game. The output might be a class score, a number, a probability distribution, a ranked list, a generated token, or an action. The practical question is whether the chosen function has the right form for the task.

**Definition 1.2.1** (Function) *Let $\mathcal{X}$ and $\mathcal{Y}$ be sets. A function $f : \mathcal{X} \to \mathcal{Y}$ assigns to each input $x \in \mathcal{X}$ exactly one output $f(x) \in \mathcal{Y}$. The set $\mathcal{X}$ is called the domain, and the set $\mathcal{Y}$ is called the codomain.*

In the digit example, $\mathcal{X}$ is the set of possible image representations, and $\mathcal{Y}$ is the set of possible outputs. If the task is binary classification, where the model predicts whether an image is a 0 or a 1, a convenient codomain is the interval $[0, 1]$. An output near 1 can be interpreted as strong evidence for class 1, while an output near 0 can be interpreted as strong evidence for class 0. The same image data would require a different codomain if the task were to predict ten digit classes, reconstruct a clean image, or generate a caption.

Deep-learning models are families of functions with adjustable parameters. The notation

$$f_\theta : \mathcal{X} \to \mathcal{Y}$$

means that $f$ is a function whose behavior depends on the parameter collection $\theta$. In a small linear model, $\theta$ may contain only a few numbers. In a modern neural network, $\theta$ may contain millions or billions of weights and biases. The mathematical role is the same: training selects parameter values that make the function useful on data.

Training changes $\theta$, so it changes which function from the family is used.

**Definition 1.2.2** (Parameterized model) *A parameterized model is a family of functions $\{f_\theta : \mathcal{X} \to \mathcal{Y}\}_{\theta \in \Theta}$, where $\Theta$ is the set of allowed parameter values. The parameter $\theta$ determines which function in the family is used for prediction.*

The two-feature example gives a complete numerical instance of the definition. Let the model be

$$f_\theta(x) = w_1x_1 + w_2x_2 + b, \qquad \theta = (w_1, w_2, b).$$

**Pause and predict**

Before substituting numbers, identify which symbols are fixed by the current example and which symbols training is allowed to change. Then predict what happens to $f_\theta(x)$ if $w_1$ increases while $x_1 > 0$.

For $x = (0.20, 0.80)$, $w_1 = 0.70$, $w_2 = 0.40$, and $b = -0.10$, the output is

$$f_\theta(x) = 0.70(0.20) + 0.40(0.80) - 0.10 = 0.36.$$

The input values 0.20 and 0.80 are observed data. The values 0.70, 0.40, and $-0.10$ are parameters. If training changes $w_1$ from 0.70 to 0.90, then the function itself has changed, even though the formula still has the same shape.

A neural network is a particular kind of parameterized model built by composing simple functions. A layer forms weighted sums and then usually applies a nonlinear operation. Stacking layers creates a composition: the output of one function becomes the input to the next. This view is central to the mathematical treatment of deep learning because it explains why the chain rule appears in training and why intermediate representations can be learned [180, 17].

**Definition 1.2.3** (Neural network) *For this book, a neural network is a parameterized composition*

$$f_\theta = L_{m,\theta_m} \circ L_{m-1,\theta_{m-1}} \circ \cdots \circ L_{1,\theta_1},$$

*where each $L_{j,\theta_j}$ is a layer map and $\theta = (\theta_1, \ldots, \theta_m)$ collects the trainable parameters. A deep neural network has multiple successive learned layer maps in this composition.*

Before optimization, check three modeling choices: input information, output space, and searchable function family. Wrong labels, discarded pixels, or a mismatched architecture can doom an experiment before training begins. Classical pattern-recognition texts and modern deep-learning references both emphasize this function-learning view, though with different notation and scope [21, 67].

## 1.3 Data, Labels, Features, and Shapes

Before training starts, the task has to be turned into data. In code this may look like loading files and arrays; mathematically it is a finite collection of examples. Each example contains the input information available at

prediction time, and in supervised learning it also carries the desired answer. The same notation will also track the shapes of the arrays that store the examples.

One case from a learning problem is an *example*. Its input is denoted by $x$. A numerical component of the input representation is a *feature*; in supervised learning, the *label* $y$ is the target value that the model should predict from $x$.

For the digit classifier, one example is one image together with its label. If the image shows a 1, then $y = 1$; if it shows a 0, then $y = 0$. The features are the pixel values after the image has been converted into numbers. The MNIST dataset is a standard historical example of this kind of handwritten-digit classification problem [126, 125]. This opening chapter does not need the full MNIST dataset, but MNIST is useful context: the same mathematical language scales from a two-feature teaching example to a large benchmark.

A supervised dataset with $n$ examples is written

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}.$$

Here $x_i$ is the input for example $i$, and $y_i$ is its label. The index $i$ is an identifier for examples, not a feature index.

The distinction between example index and feature index is small but important. In $x_i$, the subscript $i$ usually names the example. In $x_j$, when $x$ denotes the ordered list of features for one example, the subscript $j$ may name a feature. To keep the two roles visible, we often write the features of example $i$ as

$$x_i = (x_{i,1}, x_{i,2}, \ldots, x_{i,d}),$$

where $d$ is the number of features. Thus $x_{i,2}$ is the second feature of the $i$th example.

**Pause and predict**

For a dataset with 10 examples and 4 features per example, predict the shape of the data table. Then identify the entry in row 3, column 2, and name the indexing mistake that would treat that feature coordinate as a separate example.

A *scalar* is a single number, a *vector* is an ordered list of numbers, a *matrix* is a rectangular table of numbers, and a *tensor* is a multidimensional array of numbers. The shape of an array records how many entries it has along each dimension.

Consider a tiny $2 \times 2$ grayscale image,

$$\begin{bmatrix} 0.00 & 0.25 \\ 0.75 & 1.00 \end{bmatrix}.$$

As an image-shaped array, its shape is $2 \times 2$. If we flatten it into a vector by reading rows from left to right, it becomes

$$x = (0.00, 0.25, 0.75, 1.00) \in \mathbb{R}^4.$$

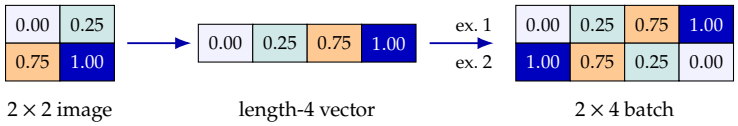


**Figure 1.4:** Flattening keeps the same pixel values but changes their shape. A batch stacks several flattened examples as rows of one matrix.

Flattening changes the shape from $2 \times 2$ to 4, but it does not change the numerical values. If we store two such flattened images in one batch, the batch has shape $2 \times 4$: two examples and four features per example.

A *batch* is a subset of examples processed together during training or evaluation. If a batch contains $m$ flattened inputs and each input has $d$ features, the input batch can be represented by a matrix $X \in \mathbb{R}^{m \times d}$.

Shape errors are mathematical errors: the objects no longer match the formulas.

Shape reasoning is an applied skill. In practice, many errors occur because an array has the wrong shape. A model expecting $m \times 64$ flattened examples cannot directly receive an $m \times 8 \times 8$ image tensor unless the model or preprocessing step is designed for that shape. Similarly, a model that outputs one number per example for binary classification has a different output shape from a model that outputs ten class scores. Reading shapes is one of the first practical mathematical skills in deep learning.

Numerical scale is another early practical issue. Pixel values are often stored as integers between 0 and 255. Dividing by 255 maps them to the interval $[0, 1]$. For example, a pixel value of 64 becomes

$$64/255 \approx 0.251.$$

This normalization does not change which pixels are darker or lighter, but it changes the numerical scale seen by the optimizer. Later chapters will discuss optimization more carefully; for now, the key point is that data representation is already a mathematical choice with practical consequences.

## 1.4 Linear Algebra for Shape Reasoning

Deep-learning formulas use linear algebra constantly, but the first goal is not to memorize every topic from a linear algebra course. The practical goal is to read an expression, identify the shape of each object, and decide whether the operation is meaningful. This section collects the small set of tools needed for the early chapters: dot products, matrix-vector products, affine maps, batches, transposes, and norms.

**Definition 1.4.1** (Dot product) *For two vectors $w, x \in \mathbb{R}^d$, the dot product is*

$$w^\top x = \sum_{j=1}^{d} w_j x_j.$$

*It multiplies corresponding entries and adds the results, producing one scalar.*

For a two-feature digit example with $w = (0.30, 0.40)$ and $x = (0.20, 0.80)$,

$$w^\top x = 0.30(0.20) + 0.40(0.80) = 0.38.$$

The dot product is a weighted sum of features. A positive weight increases the score when the corresponding feature is large; a negative weight decreases it. The transpose in $w^\top x$ marks the usual convention that $x$ is treated as a column vector and $w^\top$ as a row vector.

**Definition 1.4.2** (Matrix-vector product) *If $W \in \mathbb{R}^{k\times d}$ and $x \in \mathbb{R}^d$, then $Wx \in \mathbb{R}^k$. Each entry of $Wx$ is a dot product between one row of $W$ and the vector $x$:*

$$(Wx)_i = \sum_{j=1}^{d} W_{i,j} x_j.$$

Thus a matrix-vector product computes several weighted sums at once. If

$$W = \begin{bmatrix} 1 & 2 \\ -1 & 3 \\ 0 & 4 \end{bmatrix}, \qquad x = \begin{bmatrix} 0.20 \\ 0.80 \end{bmatrix},$$

then $W \in \mathbb{R}^{3\times 2}$, $x \in \mathbb{R}^2$, and

$$Wx = \begin{bmatrix} 1.80 \\ 2.20 \\ 3.20 \end{bmatrix} \in \mathbb{R}^3.$$

The output has three entries because $W$ has three rows.

**Pause and predict**

Suppose $W \in \mathbb{R}^{4\times 7}$, $x \in \mathbb{R}^7$, and $b \in \mathbb{R}^4$. Predict the shape of $Wx$, then predict whether $Wx + b$ is shape-compatible. Which dimension of $W$ determines the number of output features, and what mismatch would appear if $b$ had length 7 instead?

**Definition 1.4.3** (Affine map) *An affine map first applies a linear map and then adds a bias:*

$$z = Wx + b.$$

*If $W \in \mathbb{R}^{k\times d}$, $x \in \mathbb{R}^d$, and $b \in \mathbb{R}^k$, then $z \in \mathbb{R}^k$.*

The word *linear* is often used informally for layers of the form $Wx + b$, but the bias makes the map affine rather than strictly linear. Neural-network code often calls this a linear or dense layer because the main parameter array is the weight matrix $W$. The shape check is the important habit: $W$'s column count must match the length of $x$, and the bias must match the output length.

A valid matrix product makes the inner dimensions match and leaves the outer dimensions.

Batches add one more shape convention. Earlier, a batch of $m$ examples with $d$ features each was stored as $X \in \mathbb{R}^{m\times d}$, with one example per row. For a

dense layer whose single-example formula is $z = Wx + b$ with $W \in \mathbb{R}^{k\times d}$, the batched score matrix is commonly written

$$Z = XW^\top + \mathbf{1}_m b^\top.$$

Here $XW^\top \in \mathbb{R}^{m\times k}$. The vector $\mathbf{1}_m$ contains $m$ ones, so $\mathbf{1}_m b^\top \in \mathbb{R}^{m\times k}$ repeats the same bias row for every example in the batch. In code this repetition is usually handled by broadcasting.

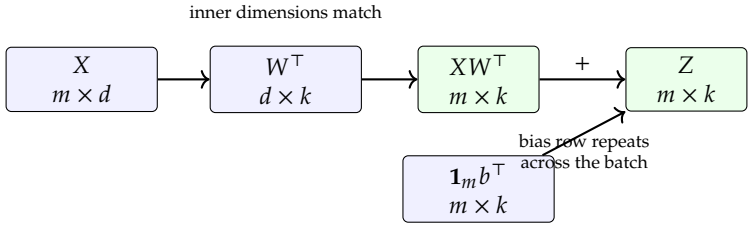


**Figure 1.5:** The batched affine map keeps the example axis $m$, matches the feature axis $d$, and produces one $k$-dimensional score vector per example.

**Definition 1.4.4** (Transpose) *The transpose of a matrix swaps its rows and columns. If $W \in \mathbb{R}^{k\times d}$, then $W^\top \in \mathbb{R}^{d\times k}$. For a vector $x$, the notation $x^\top$ turns a column vector into a row vector.*

Transposes are not decoration; they make shapes line up. For example, later backpropagation formulas use a vector $g_z \in \mathbb{R}^k$ with the same shape as $z = Wx + b$ to produce

$$W^\top g_z \in \mathbb{R}^d, \qquad g_z x^\top \in \mathbb{R}^{k\times d}.$$

The first expression has the shape of the input $x$. The second has the shape of the weight matrix $W$. Chapter 5 derives these formulas; for now, the point is that shape reasoning already predicts what kind of object each gradient must be.

**Definition 1.4.5** (Euclidean norm and distance) *The Euclidean norm of $x \in \mathbb{R}^d$ is*

$$\|x\|_2 = \sqrt{x_1^2 + \cdots + x_d^2}.$$

*The Euclidean distance between two vectors $x, y \in \mathbb{R}^d$ is $\|x - y\|_2$.*

Norms measure vector size, and distances compare vectors. They will matter when the book discusses representation geometry, embeddings, optimization steps, and rules that rescale unusually large gradient updates. The same shape rule applies: $x - y$ only makes sense when $x$ and $y$ have the same length.

| Expression | Required shapes | Result shape |
|---|---|---|
| $w^\top x$ | $w, x \in \mathbb{R}^d$ | scalar |
| $Wx$ | $W \in \mathbb{R}^{k\times d}$, $x \in \mathbb{R}^d$ | $\mathbb{R}^k$ |
| $Wx + b$ | $W \in \mathbb{R}^{k\times d}$, $x \in \mathbb{R}^d$, $b \in \mathbb{R}^k$ | $\mathbb{R}^k$ |
| $XW^\top$ | $X \in \mathbb{R}^{m\times d}$, $W \in \mathbb{R}^{k\times d}$ | $\mathbb{R}^{m\times k}$ |
| $g_z x^\top$ | $g_z \in \mathbb{R}^k$, $x \in \mathbb{R}^d$ | $\mathbb{R}^{k\times d}$ |
| $W^\top g_z$ | $W \in \mathbb{R}^{k\times d}$, $g_z \in \mathbb{R}^k$ | $\mathbb{R}^d$ |
| $\|x - y\|_2$ | $x, y \in \mathbb{R}^d$ | scalar |

**Remark 1.4.1** (Matrix products and outer products) Two products appear often in shape checks. If $A \in \mathbb{R}^{p\times q}$ and $B \in \mathbb{R}^{q\times r}$, then the matrix product $AB \in \mathbb{R}^{p\times r}$ has entries

$$(AB)_{i,j} = \sum_{\ell=1}^{q} A_{i,\ell} B_{\ell,j}.$$

Thus the inner dimensions must match, and the outer dimensions give the result.
For vectors $u \in \mathbb{R}^p$ and $v \in \mathbb{R}^q$, the outer product $uv^\top \in \mathbb{R}^{p\times q}$ has entries $(uv^\top)_{i,j} = u_i v_j$. The matrix product composes linear maps, while the outer product builds a matrix from two vector-shaped factors.

The book will introduce more linear algebra only when a model needs it. For the next few chapters, the essential discipline is to ask: What is the shape of each object, which dimensions must match, and what shape should the result have?

## 1.5 Predictions, Loss, and Empirical Risk

Once inputs and labels are defined, the next question is how to judge a model's output. A prediction is the value produced by the model for a given input. A loss is a numerical penalty for a prediction. The loss is crucial because training needs a scalar objective to minimize; vague statements such as "predict well" must become a number that can be computed on examples.

**Definition 1.5.1** (Prediction) *For a model $f_\theta$ and input $x$, the prediction is*

$$\hat{y} = f_\theta(x).$$

*The symbol $\hat{y}$, read "y hat," means the model's predicted value. The label $y$ is the target value supplied by the data.*

In binary digit classification, a common choice is to let $\hat{y} \in [0, 1]$. The number $\hat{y}$ can be interpreted as the model's estimated probability that the label is 1. A threshold such as 0.5 can then convert the score into a hard class decision. For example, if $\hat{y} = 0.82$, the model predicts class 1 under the 0.5 threshold. If the true label is also $y = 1$, the hard decision is correct.

**Definition 1.5.2** (Loss function) *A loss function is a function $\ell(\hat{y}, y)$ that assigns a scalar penalty to a prediction $\hat{y}$ and target $y$. Smaller loss means the prediction is better according to the chosen objective.*

For a numerical prediction problem, a common loss is squared error,

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

**Pause and predict**

With $y = 1$, predict which squared-error loss is larger: $\hat{y} = 0.80$ or $\hat{y} = 0.20$. Then predict whether the answer depends on the model being a neural network or only on the two numerical predictions.

If $y = 1$ and $\hat{y} = 0.80$, the squared error is $(0.80 - 1)^2 = 0.04$. If $y = 1$ and $\hat{y} = 0.20$, the squared error is $(0.20 - 1)^2 = 0.64$. The second prediction receives a larger penalty because it is farther from the target.

Squared error is not the only useful loss. For classification models that return probabilities, another common loss is cross-entropy [21, 67]. Its basic idea is that the model receives a small penalty when it assigns high probability to the observed label and a large penalty when it assigns low probability to that label. This chapter only needs that intuition; Chapter 3 develops cross-entropy in detail for probability vectors and one-hot labels, where a one-hot label is a class label written as a 0/1 vector.

A loss turns prediction quality into the scalar signal that training can minimize.

A dataset contains many examples, so the standard training objective is the average loss over the training dataset.

**Definition 1.5.3** (Empirical risk) *Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, model $f_\theta$, and loss $\ell$, the empirical risk is*

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

*It is the average loss of the model on the examples in $\mathcal{D}$.*

For three examples, this definition expands to

$$\widehat{R}(\theta) = \frac{\ell(f_\theta(x_1), y_1) + \ell(f_\theta(x_2), y_2) + \ell(f_\theta(x_3), y_3)}{3}.$$

The summation symbol is therefore not a new kind of operation; it is compact notation for adding the losses and dividing by the number of examples.

**Example 1.5.1** (Computing an empirical risk) Suppose the three example losses are 0.04, 0.64, and 0.09. Then the empirical risk is their average:

$$\widehat{R}(\theta) = \frac{0.04 + 0.64 + 0.09}{3} \approx 0.257.$$

The high second loss has a visible effect on the average. Training tries to change $\theta$ so that averages like this become smaller on the training data.

Loss and accuracy are related but not identical. Accuracy counts how often the hard decision is correct. Loss measures how strongly the model's numerical prediction agrees with the target. Two models can have the same accuracy but different losses if one model is more confident in the correct answers. This distinction matters in applied work because training usually optimizes a loss, while reports often include metrics such as accuracy, precision, recall, or latency. A rigorous experiment states both what was optimized and what was reported.

## 1.6 Parameters, Representations, and Layers

So far, $\theta$ has stood for "the parameters" without saying what sits inside that collection. In neural networks, the familiar pieces are weights and biases. A dense layer multiplies inputs by weights, adds biases, and usually passes the result through a nonlinear activation. Those pieces are small enough to compute with directly, but they are also the building blocks of large networks.

**Definition 1.6.1** (Weight and bias) *A weight is a parameter that multiplies an input or activation. A bias is a parameter added to a weighted sum. In a scalar linear model $z = wx + b$, $w$ is a weight and $b$ is a bias.*

For the two-feature example,

$$z = w_1 x_1 + w_2 x_2 + b.$$

The weight $w_1$ controls how strongly the first feature contributes to the score, while $w_2$ controls the second feature. The bias $b$ shifts the score even when the input values are zero. If $x = (0.20, 0.80)$, $w_1 = 0.30$, $w_2 = 0.40$, and $b = 0.10$, then

$$z = 0.30(0.20) + 0.40(0.80) + 0.10 = 0.48.$$

Changing a weight changes the model's response to the corresponding feature. Changing the bias shifts the score for all inputs.

**Definition 1.6.2** (Activation function) *An activation function is a scalar map $\sigma : \mathbb{R} \to \mathbb{R}$ applied to a layer's pre-activation values component by component. A nonlinear activation is one for which $\sigma$ is not an affine function. Common examples include*

$$\mathrm{ReLU}(z) = \max\{0, z\}, \qquad \mathrm{sigmoid}(z) = \frac{1}{1 + e^{-z}}.$$

The distinction between the pre-activation value $z$ and the activation is important. If $z = -0.40$, then $\mathrm{ReLU}(z) = 0$. If $z = 0.48$, then $\mathrm{sigmoid}(z) \approx 0.618$. For binary classification, the sigmoid function is useful because it maps any real-valued score to a number in $(0, 1)$, so the output can be read as a probability-like prediction.

**Pause and predict**

Before reading the layer definition, predict the shape of a weight matrix that maps 4 input features to 3 output values. Then predict how many bias entries are needed for those 3 outputs.

**Definition 1.6.3** (Layer) *A layer is a parameterized transformation that maps one array of numbers to another array of numbers. A dense layer with input $x \in \mathbb{R}^d$ and output $h \in \mathbb{R}^k$ has the form*

$$h = \sigma(Wx + b),$$

*where $W \in \mathbb{R}^{k \times d}$ is a weight matrix, $b \in \mathbb{R}^k$ is a bias vector, and $\sigma$ is applied componentwise.*

This definition also explains parameter count. A dense layer from 64 input features to 10 outputs has $10 \times 64 = 640$ weights and 10 biases, for a total of 650 trainable parameters. Parameter count matters because it affects memory use and computation. It also affects how many different functions the layer family can represent; Section 1.8 will call this expressive size *capacity*. A larger layer can fit a small training dataset in ways that do not persist on new examples; the same section will introduce validation and test samples as safeguards against that failure.

Neural networks become deep by composing layers. A one-hidden-layer network for the digit example can be written

$$h = \sigma(W_1 x + b_1), \qquad \hat{y} = \mathrm{sigmoid}(W_2 h + b_2).$$

The vector $h$ is called a hidden representation because it is neither the raw input nor the final output. It is an internal description learned by the model. Representation learning is one of the central reasons deep learning is useful: the model can learn intermediate features instead of relying only on features designed by hand [180, 17].

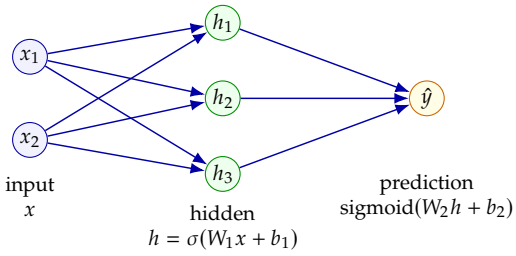


**Figure 1.6:** A one-hidden-layer network composes two parameterized transformations. The hidden activations form a learned representation before the final prediction is produced.

**Definition 1.6.4** (Representation) *A representation of an input $x$ is the value*

$$h = \phi(x) \in \mathcal{Z}$$

*produced by a representation map $\phi : \mathcal{X} \to \mathcal{Z}$. In a neural network, a hidden representation is an intermediate activation produced inside the model, such as $h = \sigma(W_1 x + b_1)$.*

The need for nonlinear activations already appears in the simplest stacked case. For two affine layers, suppose $h = W_1 x + b_1$ and $\hat{y} = W_2 h + b_2$. Substituting the first equation into the second gives

$$\hat{y} = W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2).$$

This again has the form $a \mapsto Wa + b$. Repeating the same substitution handles any finite stack of such layers.

Without nonlinear activations, stacked linear layers collapse to one linear layer.

Nonlinear activations are therefore not a cosmetic detail; they are part of what makes deep networks expressive. Universal-approximation results formalize the expressive power of neural networks under suitable assumptions [42, 91], although practical success also depends on data, optimization, architecture, and computation.

## 1.7 Training as Optimization

Training begins after the model family, dataset, and loss have been chosen. At that point the open question is which parameter values to use. The usual answer is to make the empirical risk small, so training becomes an optimization problem. The full mathematics of backpropagation and stochastic optimization will appear later in the book; here we only need a small calculus preview and the language for describing what the updates are trying to do.

**Definition 1.7.1** (Derivative and partial derivative) *For a one-parameter scalar function $J(w)$, the derivative*

$$\frac{dJ}{dw}(w) = \lim_{\epsilon \to 0} \frac{J(w+\epsilon) - J(w)}{\epsilon}$$

*when the limit exists, measures the local rate at which $J$ changes as $w$ changes. For a scalar function $J(\theta_1, \ldots, \theta_m)$, the partial derivative $\partial J / \partial \theta_j$ asks the same local question for one coordinate while the other coordinates are held fixed.*

**Definition 1.7.2** (Gradient and directional derivative) *For a differentiable scalar function $J(\theta)$, the gradient $\nabla_\theta J(\theta)$ is the vector of partial derivatives of $J$ with respect to the components of $\theta$. It points in the direction of steepest local increase under the Euclidean norm. If $d$ is a direction, the directional derivative of $J$ at $\theta$ in direction $d$ is*

$$\nabla_\theta J(\theta)^\top d.$$

*It predicts the first-order change in $J$ for a small move in direction $d$.*

The scalar chain rule will be used repeatedly. If $u = f(p)$ and $J = h(u)$ are differentiable scalar functions, then

$$\frac{dJ}{dp} = \frac{dJ}{du}\frac{du}{dp}.$$

The effect of $p$ on $J$ is the product of the local effects along the computational chain.

**Definition 1.7.3** (Optimization problem) *An optimization problem asks for an input that makes an objective function as small or as large as possible over an allowed set of inputs.*

In supervised deep learning, the objective is often the empirical risk $\widehat{R}(\theta)$, so training is commonly described as minimizing $\widehat{R}$ over an allowed parameter set. Chapter 5 names this special case empirical-risk minimization and studies how numerical methods approach it.

The notation $\arg\min_\theta J(\theta)$ means the parameter values at which the objective $J$ is minimized. It does not mean the minimum loss value itself. If several parameter values tie for the same smallest loss, arg min can contain more than one solution. In practice, deep-learning training rarely finds a perfect global minimum that can be certified mathematically. Instead, it uses iterative methods that repeatedly improve the parameters according to local information [67].

Because the gradient points toward local increase, a basic gradient-descent step moves in the negative gradient direction. The minus sign can be justified as a local statement.

A gradient is local information. The learning rate controls how far the update follows it.

**Proposition 1.7.1** (Negative gradient as a descent direction) *Let $J(\theta)$ be differentiable, and suppose $\nabla_\theta J(\theta) \neq 0$. The direction $-\nabla_\theta J(\theta)$ is a local descent direction for $J$.*

*Proof.* The directional derivative of $J$ at $\theta$ in the direction $d = -\nabla_\theta J(\theta)$ is

$$\nabla_\theta J(\theta)^\top d = -\|\nabla_\theta J(\theta)\|^2 < 0.$$

Thus a sufficiently small move in this direction decreases $J$ to first order. □

This is why a basic gradient-descent step has the form

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \widehat{R}(\theta_t).$$

Here $t$ is the iteration number, and $\eta > 0$ is the learning rate. The learning rate is a step size. If it is too small, training may be slow. If it is too large, training may jump past useful parameter values or become unstable.

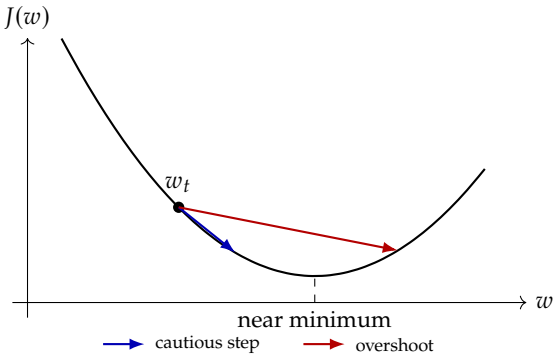


**Figure 1.7:** A gradient-descent update follows a local downhill direction. The learning rate decides whether the step is cautious or large enough to overshoot the useful part of the loss curve.

**Pause and predict**

For $w_t = 0.50$, $\eta = 0.10$, and gradient $g = -0.30$, predict whether gradient descent increases or decreases $w$. Then predict how the answer would change if the gradient were $+0.30$ instead.

**Example 1.7.1** (One-parameter update) Suppose the current parameter is $w_t = 0.50$, the learning rate is $\eta = 0.10$, and the gradient of the loss with respect to $w$ is $g = -0.30$. The update is

$$w_{t+1} = w_t - \eta g = 0.50 - 0.10(-0.30) = 0.53.$$

The negative gradient caused $w$ to increase. This does not mean parameters always increase during training; the direction depends on the current gradient.

The gradient can also be computed by hand in a tiny model. Let

$$\hat{y} = wx + b, \qquad \ell = (\hat{y} - y)^2.$$

If $x = 2$, $y = 1$, $w = 0.20$, and $b = 0.10$, then $\hat{y} = 0.50$ and $\ell = 0.25$. The derivative with respect to $w$ is

$$\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} = 2x(wx + b - y).$$

Substituting the numbers gives $2(2)(0.50 - 1) = -2$. The derivative with respect to $b$ is $2(wx + b - y) = -1$. Both derivatives are negative, so a small gradient-descent step increases both $w$ and $b$, raising the prediction toward the target $y = 1$.

**Example 1.7.2** (One batch through one update) A two-example batch can show the whole training calculation in miniature. Let

$$X = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \qquad y = \begin{bmatrix} 2 \\ 4 \end{bmatrix}, \qquad \hat{y} = Xw + \mathbf{1}b.$$

At $w = 1$ and $b = 0$, the predictions are

$$\hat{y} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Using squared loss, both examples have loss 1, so the batch empirical risk is 1. The batch gradients are

$$\frac{\partial \widehat{R}}{\partial w} = \frac{1}{2} \sum_{i=1}^{2} 2(\hat{y}_i - y_i)x_i = \frac{1}{2} (2(-1)(1) + 2(-1)(3)) = -4,$$

and

$$\frac{\partial \widehat{R}}{\partial b} = \frac{1}{2} \sum_{i=1}^{2} 2(\hat{y}_i - y_i) = -2.$$

With learning rate $\eta = 0.1$, one gradient step gives

$$w \leftarrow 1 - 0.1(-4) = 1.4, \qquad b \leftarrow 0 - 0.1(-2) = 0.2.$$

The batch, affine prediction, per-example losses, empirical risk, gradients, and update are all different views of the same training step.

Backpropagation is the efficient application of the chain rule to compute such derivatives in large composed models. The original modern presentation of learning representations by back-propagating errors is due to Rumelhart, Hinton, and Williams [180]; related gradient ideas also appear in earlier and broader forms, including work by Werbos [228]. This chapter uses backpropagation only as a conceptual preview. Later chapters will handle vector derivatives, computational graphs, minibatches, stochastic gradients, and optimizers such as Adam [110].

## 1.8 Generalization, Capacity, and Inductive Bias

Generalization is the difference between minimizing a loss on the examples in front of us and minimizing the same loss on the source that will produce future examples. In this first pass, read a population distribution as that future source and read $\mathbb{E}$ as an average over it; Chapter 3 gives the probability language more formally. If $P$ is the population distribution on input-label pairs $(X, Y)$, then the quantity of interest is the population risk

$$R(\theta) = \mathbb{E}_{(X,Y)\sim P}\left[\ell(f_\theta(X), Y)\right].$$

The training set provides only a finite sample, so training minimizes or approximately minimizes the empirical risk

$$\widehat{R}_{\mathcal{D}_{\text{train}}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{(x_i,y_i)\in\mathcal{D}_{\text{train}}} \ell(f_\theta(x_i), y_i).$$

Training loss is a sample average. Generalization asks how that average relates to a population expectation.

**Definition 1.8.1** (Training, validation, and test samples) *Training, validation, and test samples are disjoint finite samples*

$$\mathcal{D}_{\text{train}}, \qquad \mathcal{D}_{\text{val}}, \qquad \mathcal{D}_{\text{test}}.$$

*The fitted parameter $\widehat{\theta}$ is selected using $\mathcal{D}_{\text{train}}$. Model-selection choices may depend on $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{val}}$. The test sample is reserved for estimating the risk of the completed procedure after those choices are fixed.*

The split matters because the fitted parameter is random: it depends on the sample that produced it. If

$$\widehat{\theta} \in \arg\min_{\theta\in\Theta} \widehat{R}_{\mathcal{D}_{\text{train}}}(\theta),$$

then $\widehat{R}_{\mathcal{D}_{\text{train}}}(\widehat{\theta})$ is an optimistic estimate of $R(\widehat{\theta})$. The parameter was chosen partly because it performed well on that particular sample. A held-out sample gives a cleaner estimate because it did not participate in the minimization.

**Definition 1.8.2** (Generalization gap) *For a fixed parameter vector $\theta$ and dataset $\mathcal{D}$, the population generalization gap is*

$$\Delta_{\mathcal{D}}(\theta) = R(\theta) - \widehat{R}_{\mathcal{D}}(\theta).$$

*When $R(\theta)$ is unknown, the difference between validation risk and training risk is a computable empirical proxy, not the population generalization gap itself.*

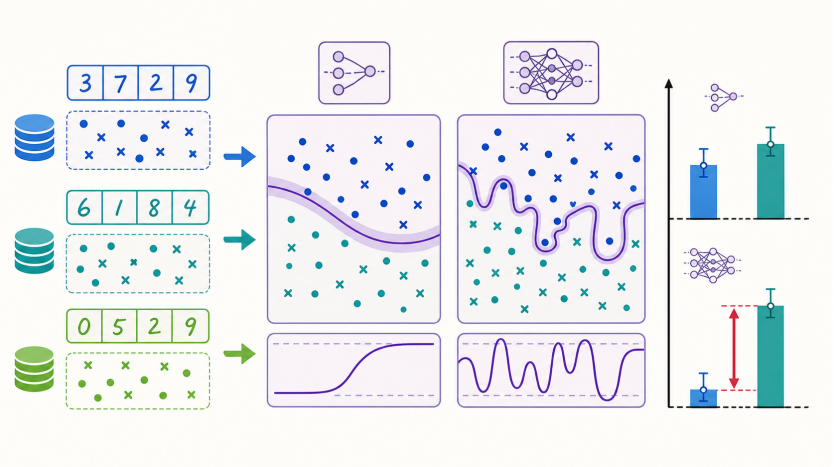


**Figure 1.8:** Training, validation, and test data have different mathematical roles. A larger model can drive the training average down while the validation average remains high, producing an empirical generalization gap.

**Pause and predict**

Suppose a model has much lower training loss than validation loss. Which term in the comparison changed: the formula for the loss, the parameter vector, or the sample used in the average? Explain why the same $\theta$ can have different empirical risks on two samples.

A small table makes the sample-dependence visible:

| Model | $\widehat{R}_{\mathcal{D}_{\text{train}}}$ | $\widehat{R}_{\mathcal{D}_{\text{val}}}$ |
|---|---|---|
| Small model | 0.18 | 0.20 |
| Very large model | 0.02 | 0.64 |

The larger model is better only if the training average is treated as the target. The validation average suggests that the parameter found inside the larger class does not estimate the population risk well.

Capacity controls how many different functions a model class can represent. Define the hypothesis class as

$$\mathcal{F}_\Theta = \{f_\theta : \theta \in \Theta\}$$

for the chosen parameter set $\Theta$. A class with larger capacity can lower the best achievable training risk, but the same flexibility can make the selected function more sensitive to sample noise.

For now, keep the qualitative message. Finite-class generalization bounds shrink with sample size and grow with candidate-class size. Chapter 3 supplies the probability details after introducing independence, sample means, and tail bounds. The shape of the result is still worth remembering: more data makes empirical averages more trustworthy, while a larger search space gives training more chances to find an accidentally good training average [220, 188].

**Definition 1.8.3** (Inductive bias) *An inductive bias is any part of the learning rule fixed before the particular training sample is observed that restricts or weights the functions that can be selected. A prior is information, a preference, or a probability distribution fixed before seeing the particular training sample. An inductive bias may enter through the hypothesis class, the parameterization, a penalty term, such a prior, or the optimization algorithm.*

Bias can enter through the class $\mathcal{F}_\Theta$, the parameterization $\theta \mapsto f_\theta$, the optimization algorithm, or an explicit penalty. For example, regularized empirical risk minimization chooses

$$\widehat{\theta}_\lambda \in \arg\min_{\theta\in\Theta} \left\{ \widehat{R}_{\mathcal{D}_{\text{train}}}(\theta) + \lambda\Omega(\theta) \right\},$$

where $\Omega(\theta)$ penalizes some parameters more than others. A convolutional architecture imposes a different bias: it restricts the function class to reuse local filters across spatial positions. The applied question "does this model fit the task?" becomes the mathematical question "does this class and penalty make the right functions easy to find from the available sample?"

## 1.9 Notation and Reading Guide

Notation should remove ambiguity, not add ceremony. The table below collects the symbols used in the chapter and gives the running-example meaning for each one. Readers do not need to memorize every symbol immediately. The useful habit is to ask what kind of object a symbol denotes: number, vector, matrix, function, parameter, dataset, loss, or gradient.

Read each symbol by its type first: data, parameter, function, loss, or update direction.

| Symbol | Meaning | Running-example interpretation |
|---|---|---|
| $x$ | input | one digit image represented by numbers |
| $y$ | label or target | 0 or 1 for the digit class |
| $\hat{y}$ | prediction | the model's output for one image |
| $f_\theta$ | parameterized model | the classifier with parameters $\theta$ |
| $\theta$ | parameter collection | all weights and biases |
| $\mathcal{D}$ | dataset | a collection of labeled digit examples |
| $\ell(\hat{y}, y)$ | loss | penalty for one prediction |
| $\widehat{R}(\theta)$ | empirical risk | average loss on a dataset |
| $\nabla_\theta$ | gradient with respect to parameters | local direction used for training |

Subscripts usually indicate either an example index or a component index. In $(x_i, y_i)$, the subscript $i$ names the $i$th example. In $x = (x_1, x_2, \ldots, x_d)$, the subscript names a feature. When both are needed at once, $x_{i,j}$ denotes feature $j$ of example $i$. The comma is only a separator between the example

index and the feature index, so $x_{3,2}$ means the second feature of the third example, not the 32nd example.

Vectors and matrices carry shape information. A vector $x \in \mathbb{R}^d$ contains $d$ real-valued features. A matrix $X \in \mathbb{R}^{m \times d}$ can contain a batch of $m$ examples, each with $d$ features. A weight matrix $W \in \mathbb{R}^{k \times d}$ maps a $d$-dimensional input vector to a $k$-dimensional output vector through $Wx$. The product $Wx$ is defined because the inner dimensions match: $d$ input features and $d$ columns in $W$.

Probability notation will appear more often in later chapters, but one idea is already present. A model output such as $\hat{y} = 0.82$ in binary classification is often interpreted as an estimated probability. The loss then uses that number to penalize disagreement with the observed label. This does not mean every model output is automatically a well-calibrated probability. Calibration is a separate question; the notation only states how the output is used by the chosen objective.

The most important reading habit is to connect formulas back to workflow. The expression $f_\theta(x)$ means a forward prediction. The expression $\ell(f_\theta(x_i), y_i)$ means one example's penalty. The expression $\widehat{R}(\theta)$ means an average penalty over data. The expression $\nabla_\theta \widehat{R}(\theta)$ means local information used to change the parameters. In code, these objects may appear as arrays, tensors, model calls, loss functions, and optimizer steps. The mathematical notation gives those software objects a stable interpretation.

## 1.10 Summary

Deep learning studies trainable parameterized functions. In the running digit example, the input $x$ is an image represented by numbers, the label $y$ states the desired answer, and the model $f_\theta$ maps the input to a prediction $\hat{y}$. The parameter collection $\theta$ contains the weights and biases that training changes.

Data are organized as examples, feature vectors, labels, datasets, and batches. Shapes describe how arrays are arranged, and shape reasoning is an early debugging skill in applied work. A tiny image can be flattened into a vector, and a batch of flattened images can be stored as a matrix. Dot products, matrix-vector products, affine maps, transposes, and norms are used mainly as shape-checked operations: $Wx + b$, $XW^\top$, and $W^\top g$ are readable once the dimensions are clear.

A loss function assigns a scalar penalty to a prediction. The empirical risk is the average loss over a dataset, and training is usually framed as minimizing that empirical risk. Gradients give local directions for changing parameters, and a learning rate controls the size of each update.

Layers compose weighted sums, biases, and nonlinear activations. Hidden layers produce representations, which are internal numerical descriptions learned by the model. Nonlinear activations matter because stacked linear layers without nonlinearities collapse to a single linear transformation.

A useful model must combine low training loss with generalization to new examples. Training, validation, and test sets have different roles, and mixing those roles weakens experimental claims. Capacity, regularization, and inductive bias help explain why model choice is an applied mathematical decision as well as a software decision.

## 1.11 Exercises

Answer the following ten questions in complete sentences unless a calculation is requested.

**Exercise 1.** In the running digit-classification example, identify the input, label, model, prediction, parameter collection, and loss. Use the notation from the chapter.

**Exercise 2.** A grayscale image has shape $8 \times 8$. What is the length of the vector after flattening? What is the shape of a batch containing 32 such flattened images?

**Exercise 3.** Let $w = (2, -1)$, $x = (3, 4)$, and $b = 0.5$. Compute $w^\top x + b$, and explain which symbols are data and which symbols are parameters.

**Exercise 4.** A binary classifier returns $\hat{y} = 0.9$ when the true label is $y = 1$. Compute the squared-error loss. Then compare it with the loss when $\hat{y} = 0.1$.

**Exercise 5.** Write the empirical risk for a dataset with three examples using summation notation. Then expand it as an average of three losses.

**Exercise 6.** A dense layer maps 64 inputs to 10 outputs. State the shape of $W$, compute the number of weights, the number of biases, and give the total number of trainable parameters.

**Exercise 7.** Explain why a network with only linear layers and no nonlinear activation is still just a linear function of its input.

**Exercise 8.** In one or two sentences, explain what a gradient reports during training. Then compute the update $w_{t+1} = w_t - \eta g$ for $w_t = 1.2$, $\eta = 0.05$, and $g = 4$.

**Exercise 9.** A model has 99% training accuracy but 72% validation accuracy. Name the likely issue and propose one practical response.

**Exercise 10.** Give one example of an inductive bias that is useful for images, text, or time series. Explain why it helps for that kind of data.

## 1.12 Homework: Symbolic Foundations

### Required Work

This homework uses SymPy [145] to differentiate simple losses and check the derivative formulas used in training.

### SymPy Survival Guide

SymPy is used here only as exact algebra written in Python syntax. The homework never asks for a full computer-algebra workflow. The small API below is enough for the first assignment and remains enough for most later symbolic checks:

| Mathematical action | SymPy pattern |
|---|---|
| Create symbols $x, w, b, y$ | `x, w, b, y = sp.symbols("x w b y")` |
| Build an expression $wx + b$ | `y_hat = w*x + b` |
| Square a term | `loss = (y_hat - y)**2` |
| Differentiate with respect to $w$ | `sp.diff(loss, w)` |
| Substitute values | `loss.subs(values)` |
| Simplify or expand an expression | `sp.simplify(expr)` or `sp.expand(expr)` |
| Print a decimal approximation | `sp.N(expr)` |

A SymPy expression is still an algebraic object, not a floating-point estimate. Use `sp.Rational(1, 5)` when an exact value $1/5$ is clearer than the decimal `0.2`. For matrix homework, the same rules apply after creating matrices with `sp.Matrix([...])`; ordinary multiplication uses `*`, and transposes use `.T`.

**Submission model.** Submit the completed starter script, the main hand derivation, the SymPy output needed to verify it, and a short interpretation. External computer-algebra tools may be used only as optional sanity checks.

**Task 1.** Define symbols for $x$, $w$, $b$, and $y$.

**Task 2.** Construct the one-input prediction $\hat{y} = wx + b$.

**Task 3.** Construct the squared-error loss $\ell = (\hat{y} - y)^2$.

**Task 4.** Use SymPy to compute $\partial\ell/\partial w$ and $\partial\ell/\partial b$.

**Task 5.** Substitute $x = 2$, $y = 1$, $w = 0.20$, and $b = 0.10$. Record the prediction, loss, and two derivatives.

**Task 6.** Repeat the construction for a two-input model $\hat{y} = w_1x_1 + w_2x_2 + b$.

**Task 7.** Build the average squared-error loss for two examples: $(x_1, x_2, y) = (0.20, 0.80, 1)$ and $(0.90, 0.10, 0)$.

**Task 8.** Differentiate the average loss with respect to $w_1$, $w_2$, and $b$.

**Task 9.** Substitute $w_1 = 0.30$, $w_2 = 0.40$, and $b = 0.10$. Record the average loss and the three derivatives.

**Task 10.** Write one paragraph explaining how the signs of the derivatives indicate the direction of a small gradient-descent update.

## Reference Values for Checking

For the one-input model, SymPy should produce

$$\frac{\partial \ell}{\partial w} = 2x(b + wx - y), \qquad \frac{\partial \ell}{\partial b} = 2b + 2wx - 2y.$$

With $x = 2$, $y = 1$, $w = 0.20$, and $b = 0.10$, the prediction is 0.50, the loss is 0.25, $\partial\ell/\partial w = -2.00$, and $\partial\ell/\partial b = -1.00$.

For the two-example model, with $w_1 = 0.30$, $w_2 = 0.40$, and $b = 0.10$, the average loss is 0.21925. The derivatives are approximately

$$\frac{\partial \widehat{R}}{\partial w_1} = 0.265, \qquad \frac{\partial \widehat{R}}{\partial w_2} = -0.375, \qquad \frac{\partial \widehat{R}}{\partial b} = -0.110.$$

A small gradient-descent step would decrease $w_1$, increase $w_2$, and increase $b$, because gradient descent subtracts the gradient.